

Individuell Labb 2k5: DevOps, CI/CD & Applikationssäkerhet

Översikt över labben

I denna labb kommer du att bygga vidare på din Spring Boot-applikation (från Labb 1) genom att implementera ett professionellt DevOps-arbetsflöde och genomföra en djupgående säkerhetsanalys. Du kommer att containerisera applikationen, automatisera tester och distribution via CI/CD, samt identifiera och åtgärda kritiska säkerhetsbrister baserat på OWASP Top 10.

Förkunskapskrav:

- En fungerande Spring Boot-applikation (gärna från Labb 1).
- Docker installerat lokalt.
- Ett GitHub-konto och ett konto på Docker Hub (eller GitHub Container Registry).

Steg 1: Containerisering (Docker)

Koncept: För att säkerställa att applikationen körs likadant i alla miljöer paketerar vi den i en Docker-container.

Vad du ska göra:

1. Skapa en `Dockerfile` i roten av ditt projekt.
2. Använd en "multi-stage build" för att först bygga projektet med Maven/Gradle och sedan köra det med en lättviktig JRE-image (t.ex. `eclipse-temurin:21-jre-alpine`).
3. Bygg din Docker-image lokalt med kommandot `docker build -t min-spring-app .`
4. Starta containern och säkerställ att applikationen svarar på anrop (glöm inte att skicka med miljövariabler/API-nycklar via `-e` flaggan).

✓ **Förväntat resultat:** En välkommenterad `Dockerfile`. Applikationen kan byggas och köras felfritt som en lokal Docker-container utan att Java är installerat på värdmaskinen.

Steg 2: Continuous Integration (Automatiska tester)

Koncept: Varje gång kod pushas till repositoryt måste vi garantera att befintlig funktionalitet inte går sönder.

Vad du ska göra:

1. Skapa mappen `.github/workflows/` i ditt repository.
2. Skapa filen `ci-cd-pipeline.yml`.
3. Konfigurera ett jobb (`build-and-test`) som triggas vid `push` och `pull_request` till `main` -branchen.
4. Jobbet ska checka ut koden, sätta upp Java 21, och köra dina enhetstester via `mvn test` (eller motsvarande för Gradle).

✓ **Förväntat resultat:** När du pushar kod till GitHub ser du en grön bock (Action) som indikerar att alla tester passerat automatiskt.

Steg 3: Continuous Deployment & Secrets (Säker CI/CD)

Koncept: Automatisera paketering och publicering av din applikation utan att kompromettera säkerheten (inga hårdkodade lösenord!).

Vad du ska göra:

1. Lägg till dina inloggningsuppgifter för Docker Hub (användarnamn och token) i **GitHub Secrets**.
2. Utöka din pipeline i `ci-cd-pipeline.yml` med ett nytt jobb (eller steg) som loggar in på Docker Hub.
3. Konfigurera pipelinen att bygga din Docker-image och publicera (push) den till ditt Docker-register.
4. (Frivilligt/G-krav) Lägg till ett fiktivt eller faktiskt deployment-steg (t.ex. anropa en webhook eller distribuera till en plattform). Det viktiga är att steget är *helt automatiserat*.

✓ **Förväntat resultat:** Vid en godkänd kod-push publiceras automatiskt en ny, taggad Docker-image till ditt register, och inga lösenord är synliga i loggarna.

Steg 4: Säkerhetsanalys & OWASP Top 10 (Identifiera)

Koncept: Att bygga funktionell kod räcker inte; den måste vara motståndskraftig mot attacker.

Vad du ska göra:

1. Genomför en säkerhetsgranskning av din kodbas baserat på OWASP Top 10.
2. Identifiera **minst tre** relevanta sårbarheter (eller potentiella riskområden om din Labb 1 var väldigt enkel). Exempel kan vara:
 - *A01: Broken Access Control* (t.ex. obehöriga kan komma åt känsliga endpoints).
 - *A06: Vulnerable and Outdated Components* (integrera OWASP Dependency-Check Maven-plugin i din pom.xml för att skanna efter sårbara bibliotek).
 - *A07: Identification and Authentication Failures* (t.ex. implementering av JWT för att skydda API:et).

Steg 5: Åtgärda Sårbarheterna (Implementering)

Koncept: Omsätt teori i praktik genom att stänga säkerhetshålen.

Vad du ska göra:

1. Implementera kodändringar för att åtgärda de tre identifierade sårbarheterna.
2. Om du lade till OWASP Dependency-Check, konfigurera den i CI-pipelinen så att bygget misslyckas om kritiska sårbarheter hittas.
3. Om du implementerade JWT/OAuth2, säkerställ att endpoints är korrekt skyddade.

✓ **Förväntat resultat:** Applikationen innehåller nu synliga säkerhetsmekanismer (t.ex. JWT-filter, dependency-scanning) som åtgärdar bristerna.

Steg 6: Skriftlig Säkerhetsrapport (Krav för VG)

Koncept: Att kunna motivera säkerhetsbeslut är centralt för en modern utvecklare.

Vad du ska göra:

Skriv en utförlig Markdown- eller PDF-rapport som innehåller:

1. **Identifiering:** Vilka tre sårbarheter hittade du/fokuserade du på? Hänvisa till specifika OWASP Top 10-kategorier.
2. **Åtgärd:** Hur löste du dem tekniskt i din kod?
3. **Analys & Prioritering:** Varför var dessa åtgärder viktiga? Rapporten ska visa på analytisk förmåga utöver en enkel checklista – förklara riskerna (t.ex. "Om jag inte skyddat API:et med JWT kunde vem som helst ha tömt vår AI-budget").



Bedömningskriterier (Grading Principles)

Baserat på kursplanen "Mål 5 och Mål 6" gäller följande kriterier för denna uppgift:

G Godkänd

Mål 5: Applikationen är containeriserad. En enklare CI-pipeline finns på plats som bygger koden. Applikationen fungerar i en container-miljö.

Mål 6: Studenten har identifierat och åtgärdat grundläggande säkerhetsbrister och kan redogöra för vilka förändringar som gjorts kopplat till säkerhet.

VG Väl Godkänd

För att uppnå VG krävs **hög precision** i både CI/CD och säkerhetsarbete:

- **Mål 5 (Containerisering och CI/CD):** Pipelinen inkluderar automatiserade tester, bygger och publicerar Docker-image, och genomför deployment utan manuella steg. Konfigurationen (Dockerfile & GitHub Actions) är välstrukturerad, kommenterad och reproducerbar. Hemligheter hanteras säkert via plattformens vault (t.ex. GitHub Secrets).
- **Mål 6 (Säkerhetsanalys):** Minst tre relevanta sårbarheter identifieras och åtgärdas med direkt hänvisning till OWASP Top 10 eller motsvarande ramverk. Den skriftliga rapporten visar analytisk förmåga utöver checklista-nivå och motiverar trovärdigt prioriteringen av åtgärder.

Notera: För VG i kursen som helhet krävs att den studerande uppnår VG-nivå i både Individuell Labb 1 och Individuell Labb 2k5.

Individuell Labb 2k5: DevOps, CI/CD & Application Security

Lab Overview

In this lab, you will expand upon your Spring Boot application (from Lab 1) by implementing a professional DevOps workflow and conducting an in-depth security analysis. You will containerize the application, automate testing and deployment via CI/CD, and identify and resolve critical security vulnerabilities based on the OWASP Top 10.

Prerequisites:

- A working Spring Boot application (preferably from Lab 1).
- Docker installed locally.
- A GitHub account and an account on Docker Hub (or GitHub Container Registry).

Step 1: Containerization (Docker)

Concept: To ensure the application runs identically across all environments, we package it inside a Docker container.

What to do:

1. Create a `Dockerfile` in the root of your project.
2. Use a "multi-stage build" to first build the project with Maven/Gradle and then run it using a lightweight JRE image (e.g., `eclipse-temurin:21-jre-alpine`).
3. Build your Docker image locally using `docker build -t my-spring-app .`
4. Start the container and verify that the application responds to requests (remember to pass environment variables/API keys via the `-e` flag).

✓ **Expected Output:** A well-commented `Dockerfile`. The application can be built and run flawlessly as a local Docker container without Java installed on the host machine.

Step 2: Continuous Integration (Automated Tests)

Concept: Every time code is pushed to the repository, we must guarantee that existing functionality hasn't broken.

What to do:

1. Create the `.github/workflows/` directory in your repository.
2. Create a file named `ci-cd-pipeline.yml`.
3. Configure a job (`build-and-test`) that triggers on `push` and `pull_request` to the `main` branch.
4. The job should check out the code, set up Java 21, and run your unit tests via `mvn test` (or the Gradle equivalent).

✓ **Expected Output:** When you push code to GitHub, you see a green checkmark (Action) indicating that all tests have passed automatically.

Step 3: Continuous Deployment & Secrets (Secure CI/CD)

Concept: Automate the packaging and publishing of your application without compromising security (no hardcoded passwords!).

What to do:

1. Add your Docker Hub credentials (username and token) into **GitHub Secrets**.
2. Expand your pipeline in `ci-cd-pipeline.yml` with a new job (or step) that logs into Docker Hub.
3. Configure the pipeline to build your Docker image and push it to your Docker registry.
4. (Optional/Pass Requirement) Add a mock or actual deployment step (e.g., triggering a webhook or deploying to a platform). The critical part is that the step is *entirely automated*.

✓ **Expected Output:** Upon a successful code push, a new, tagged Docker image is automatically published to your registry, and no passwords are visible in the logs.

Step 4: Security Analysis & OWASP Top 10 (Identification)

Concept: Building functional code is not enough; it must be resilient against attacks.

What to do:

1. Conduct a security review of your codebase based on the OWASP Top 10.
2. Identify **at least three** relevant vulnerabilities (or potential risk areas if your Lab 1 was very simple). Examples include:
 - *A01: Broken Access Control* (e.g., unauthorized users can access sensitive endpoints).

- *A06: Vulnerable and Outdated Components* (integrate the OWASP Dependency-Check Maven plugin in your pom.xml to scan for vulnerable libraries).
- *A07: Identification and Authentication Failures* (e.g., implementing JWT to protect the API).

Step 5: Fixing the Vulnerabilities (Implementation)

Concept: Put theory into practice by closing the security gaps.

What to do:

1. Implement code changes to resolve the three identified vulnerabilities.
2. If you added the OWASP Dependency-Check, configure it in the CI pipeline so the build fails if critical vulnerabilities are found.
3. If you implemented JWT/OAuth2, ensure the endpoints are correctly protected.

✓ **Expected Output:** The application now contains visible security mechanisms (e.g., JWT filters, dependency scanning) that fix the flaws.

Step 6: Written Security Report (Requirement for VG)

Concept: Being able to justify security decisions is central for a modern developer.

What to do:

Write a detailed Markdown or PDF report containing:

1. **Identification:** Which three vulnerabilities did you find/focus on? Reference specific OWASP Top 10 categories.
2. **Mitigation:** How did you technically resolve them in your code?
3. **Analysis & Prioritization:** Why were these measures important? The report should demonstrate analytical ability beyond a simple checklist—explain the risks (e.g., "If I hadn't protected the API with JWT, anyone could have depleted our AI budget").

Grading Principles (Bedömningskriterier)

Based on the course syllabus "Mål 5 and Mål 6", the following criteria apply for this assignment:

G Godkänd (Pass)

Mål 5: The application is containerized. A simple CI pipeline is in place that builds the code. The application functions in a containerized environment.

Mål 6: The student has identified and fixed basic security flaws and can describe the security-related changes made to the application.

VG Väl Godkänd (Pass with Distinction)

To achieve VG, **high precision** is required in both CI/CD and security work:

- **Mål 5 (Containerization and CI/CD):** The pipeline includes automated testing, builds and publishes a Docker image, and executes deployment without manual steps. The configuration (Dockerfile & GitHub Actions) is well-structured, commented, and reproducible. Secrets are managed securely via the platform's vault (e.g., GitHub Secrets).
- **Mål 6 (Security Analysis):** At least three relevant vulnerabilities are identified and fixed with direct references to the OWASP Top 10 or an equivalent framework. The written report demonstrates analytical ability beyond a checklist level and convincingly justifies the prioritization of the measures.

Note: Achieving VG on both Individuell Labb 1 and Individuell Labb 2k5 is required to be eligible for a final VG grade in the overall course.